

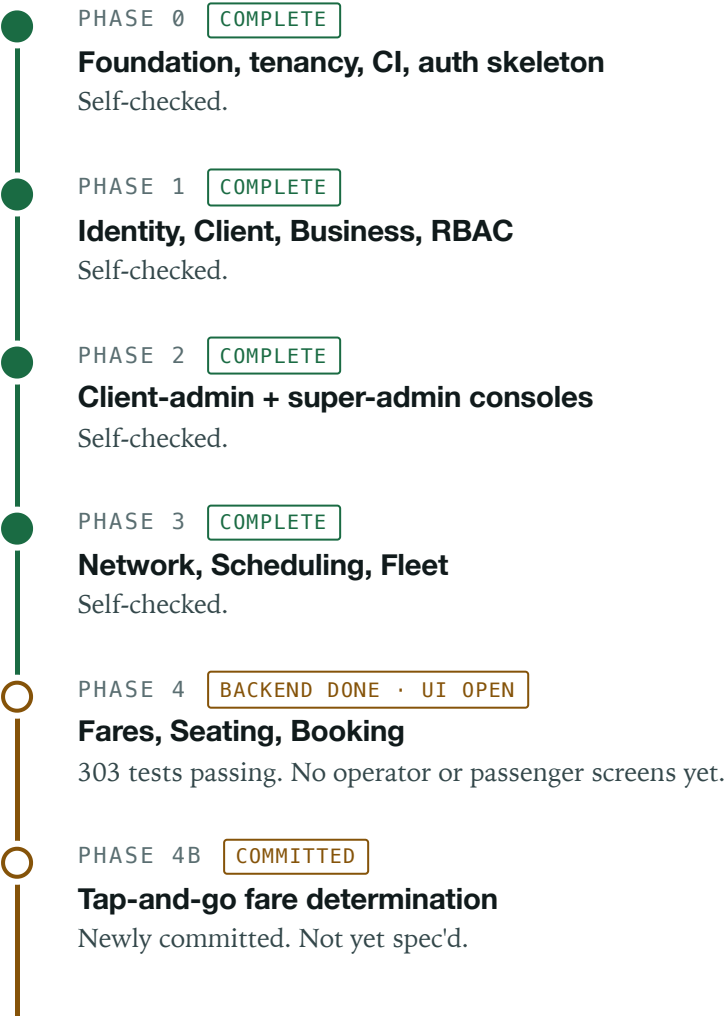
Report

The platform can model an operator's entire physical network and sell a reserved seat on it. It cannot yet take payment — and that is now the only thing waiting on a decision outside the engineering team.

303 BACKEND TESTS PASSING	97% BACKEND COVERAGE	1 OPEN DECISION BLOCKING WORK
--	-----------------------------------	--

01 Where we are

Phases 0 through 3 are complete and self-checked. Phase 4's backend is complete. Two of the three architectural decisions gating later phases are now signed off, leaving payment partner selection as the only external blocker.





PHASE 5 **BLOCKED**

Payments, Wallet, Ledger

Waiting on payment partner selection.

PHASE 6 **SEQUENCED**

QR ticketing + validator

Technically unblocked; scheduled after Phase 5.

02 Shipped

Client onboarding, self-service. An operator — Shuttlers, GUO, Cross Country, NRC — registers on the platform without us in the loop. They upload KYC on their contact person and KYB on the business itself; both land in a super-admin review queue with approve/reject and reason capture.

Multi-business under one client. A single client can run several businesses — NRC could operate an intercity arm and a metro arm separately. Each business carries its own currency, timezone, vertical, booking mode and fare pricing mode. Everything below — routes, stops, vehicles, drivers, schedules, fares, seats — is scoped to a business, so the two never see each other's data.

Network and fleet. Routes, stops, and route-stop sequencing; vehicle types, vehicles, and drivers. All manageable from the client-admin console.

Scheduling. An operator defines a recurring schedule against a route; a nightly job materialises the actual trips from it. Trips can be assigned a vehicle and driver and moved through their lifecycle.

Roles and permissions. Real DB-backed RBAC, not hardcoded checks. Three presets per client (Owner / Manager / Staff), created automatically at registration. A client admin invites their own staff and scopes what those staff can reach. Every endpoint above is individually permission-gated.

White-labelling. Per-client domain, branding, sender identity and terms; the login experience resolves from the domain the user arrived on.

Fares. Two pricing models per business: flat per-route, or per-segment (stop-pair) pricing. Operator-selectable.

Seat inventory and reservation. Real per-seat inventory per vehicle type, with per-segment availability — seat 4B can be sold Lagos→Ibadan and again Ibadan→Abuja on the same trip without conflict. Seats are held for a configurable window (default 15 minutes) and released automatically if the booking doesn't complete.

Booking. Create, cancel, and list bookings, with idempotency protection so a retried request can't produce a duplicate booking.

-
- **Concurrent seat sales cannot double-book.** This was the highest technical risk in the product. It is now enforced at the database level and verified by a test that fires six workers at the same seat simultaneously. The design decision was only accepted *after* that test passed.
 - **Tenant isolation is enforced twice** — once in the application and again by Postgres Row-Level Security. A bug in one layer does not leak one operator's data to another.

03 In flight

Operator-side screens for fares, seats and bookings. The engine is built and tested; the admin console has no pages for it yet. An operator can't set a fare or view bookings without hitting the API directly. This is the immediate gap.

Passenger app, through to reservation. The customer app is currently a shell — sign-in only. The target is search → trip selection → seat selection → hold → booking, stopping short of payment. The booking API this needs already exists and is tested, so this is UI work against a settled contract rather than new system design.

04 Committed next: tap-and-go

Tap-and-go is being promoted from a configuration flag to a real vertical. Today the mode is selectable on a business and carried on every trip, but the only code path behind it is a rejection — the booking API refuses any trip not in reservation mode. That changes.

Scope of this slice: fare determination, not fare collection. The backend will record board and alight taps against a trip, resolve the travelled segment against the per-segment fare rules already built in Phase 4, and produce a fare-owed record. It stops there. The debit itself — against a wallet balance or a card on file — belongs to Phase 5 and is not being pre-empted, because building a provisional balance now would contradict the double-entry ledger design we just signed off and guarantee rework.

Put plainly: after this slice a tap-and-go trip is fully *measured and priced* but not yet *settled*.

The validator problem, and how we're routing around it. Tap-and-go assumes a device at the vehicle. The Flutter validator app is a later phase, and waiting on it would leave this backend untested against any real client. Instead we'll build a lightweight web harness — a page with buttons standing in for each action a validator would take (tap in, tap out, trip start, trip end) — driving the real API. That gives us a demonstrable end-to-end tap-and-go flow now, and it means the validator app, when built, implements a contract that has already been exercised rather than one designed on paper.

One open question this slice must answer: what a passenger actually presents at the validator — a physical NFC card, a QR shown from the passenger app, or something else. Nothing in the system models a passenger credential today. This has commercial consequences beyond engineering — card issuance, distribution and replacement are a real operational cost — so it will be settled explicitly in the tap-and-go spec rather than defaulted into.

05 Architectural decisions now signed off

QR ticket signing. Asymmetric signing via Ed25519, with a key-id in the payload from day one so key rotation never becomes a ticket format change. Chosen over shared-secret HMAC because offline validation must eventually work on a device that can't be trusted to hold a shared secret. Phase 6's spec still has to resolve one genuine tension: a signature alone cannot express "this ticket was cancelled after issuance," which pulls against pure offline validation. That is on the record as an open question, not glossed.

Ledger chart of accounts. Double-entry from day one — every value movement is balanced journal entries, and balances are derived from the ledger rather than stored as an authoritative mutable number. Account taxonomy covers per-passenger wallets, per-business clearing, platform commission, PSP suspense, and refund/chargeback contras. Two residual questions — how commission is split across journal lines, and how PSP suspense accounts are modelled per provider — are partly shaped by which payment partner we pick, so they close alongside that decision.

Both sign-offs unblock spec work that was queued behind them. Neither closes every sub-question; that is by design — the decision records settle direction, the phase specs settle mechanics.

06 Still deferred

Payments. Nothing in the system moves money, and this is deliberate and contained: a booking terminates in `pending_payment` and goes no further. We need a partner decision — Flutterwave, Paystack, or SecurePay — before this can be spec'd, because settlement model, payout timing, refund semantics and fee structure differ enough between them to change the data model, not just the integration code. Groundwork that survives whichever we pick — idempotency keys, audit logging — is already built.

Ledger, transaction history, reporting dashboards. Design is settled; these follow payments, because a ledger with no money flowing through it has nothing to record.

QR ticketing and the Flutter validator app. Design is settled; sequenced after payments. The web harness described above covers the demonstration need in the meantime.

WHAT WE NEED FROM STAKEHOLDERS

One decision: the payment partner.

With both architectural sign-offs in hand, this is the only external blocker on the project. Phase 5, tap-and-go settlement, the ledger, reporting and payout reconciliation all queue behind it — and two open sub-questions in the chart of accounts cannot close until it is made.

07 Known gaps, for completeness

Phase 4 has not yet had its formal self-check; the 303-test figure is from implementation notes, not an independent audit. Last audited figure was 219 (Phase 3).

KYC/KYB uploads write to local disk; production object storage is documented but not built.

Production reverse-proxy topology for white-labelled custom domains is specified but not provisioned.

Platform-staff permissions are a coarse all-access flag; per-permission granularity exists only on the client side.

Integra AFC – Automated Fare Collection

Reporting period ending 10 August 2026

Companion documents: [architecture.md](#) · [decision records 0001–0006](#)